

EE345 Mod3-Lec4: Kernel Methods

References: no textbook reference

Motivation for Kernels in Regression

Recall typical setup for data fitting/regression (**Module 2**):

- features $x_i \in \mathbb{R}^d$, output $y_i \in \mathbb{R}$, $i = 1, \dots, n$
- goal: find predictor function $f(x)$ such that $f(x_i) \approx y_i$, where

$$f(x) = \sum_{i=1}^p \theta_i \phi_i(x)$$

is **linear** in parameters $\theta \in \mathbb{R}^p$, and $\phi_i(x)$ are **feature maps**

- In particular, $\phi(x) = (\phi_1(x), \dots, \phi_p(x))$ is a vector of feature maps.

Motivation for Kernels in Regression

- The predictor can be *nonlinear in data* via feature maps $\phi_i(x)$ (cf. **Module 2**)

$$x \mapsto 1, x, x^2, \dots \quad \text{or} \quad x \mapsto \max\{0, x - c_i\}$$

- We'd like to make adding new nonlinear features a bit more “automatic” $\implies$ **kernel methods**
- Basic structure:
 - ▶ Each row in the data matrix $X \in \mathbb{R}^{n \times d}$ is a sample.
 - ▶ ϕ **lifts** each row from $\mathbb{R}^d$ to $\mathbb{R}^p$
 - ▶ The kernel matrix is $K \in \mathbb{R}^{n \times n}$, comparing every pair of rows (samples)

Polynomial Kernel Example

Data: $n = 3$ samples in $d = 2$ dimensions:

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \end{bmatrix}$$

Polynomial features up to degree 2 gives $p = 5$ features:

Feature Lifting and the Kernel Trick

- The input feature vector is: $x = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d$.
- A **feature map** lifts x into a higher-dimensional space:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p, \quad \phi(x) = (\phi_1(x), \phi_2(x), \dots, \phi_p(x))^\top,$$

where typically $p \geq d$ (and possibly $p = \infty$).

- The model is **linear in feature space**:

$$f(x) = \sum_{i=1}^p \theta_i \phi_i(x) \quad \text{or equivalently} \quad f(x) = \theta^\top \phi(x), \quad \text{where } \theta \in \mathbb{R}^p$$

- The **kernel trick** avoids explicit feature lifting:

The Kernel Trick

- The feature dimension p can be huge (or infinite).
- Instead of computing $\phi(x)$ explicitly, use a kernel function:

$$K(x, x') = \phi(x)^\top \phi(x').$$

- Learning depends only on **inner products** in feature space.

Representer Theorem

The optimal f lies in the span of training features

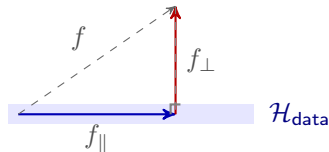
$$f(x) = \sum_{i=1}^n \alpha_i K(x, x_i),$$

where $\alpha \in \mathbb{R}^n$ are the coefficients.

Representer Theorem: Intuition

Goal:

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2.$$



Key insight:

- The loss depends on f **only through its values on training data**: $f(x_1), \dots, f(x_n)$.
- Any part of f orthogonal to $\{\phi(x_1), \dots, \phi(x_n)\}$ doesn't change these values; it only increases $\|f\|_{\mathcal{H}}^2$.

Polynomial Kernel Example

Data: $n = 3$ samples in $d = 2$ dimensions:

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ -1 & 1 \end{bmatrix}$$

Polynomial features up to degree 2 gives $p = 5$ features:

$$\phi(x) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)$$

so that

$$f(x) = \theta^\top \phi(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1 x_2 + \theta_5 x_2^2.$$

Kernel Trick (Example Continued)

Feature vectors: $\phi(x_1) = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \\ 4 \end{bmatrix}$, $\phi(x_2) = \begin{bmatrix} 3 \\ 0 \\ 9 \\ 0 \\ 0 \end{bmatrix}$, $\phi(x_3) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$

Kernel: $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$

Compute kernel matrix:

From Vanilla Features to Kernels: How to exploit the kernel trick?

- Recall that we have $XX^\top \in \mathbb{R}^{n \times n}$ and $X^\top X \in \mathbb{R}^{d \times d}$ where
- With feature maps $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ we define a new matrix of transformed features:

Covariance or Inner Product

- If we compute

$$\Phi^{\top} \Phi \in \mathbb{R}^{p \times p}$$

this is the covariance (or inner product) matrix of features, not the kernel over samples.

- It is $p \times p$ since now we are comparing features to each other.
- This is exactly why in kernel methods we usually work with $K = \Phi \Phi^{\top}$; i.e. we care about sample similarities not feature similarities.

Kernel Method

- **Kernel matrix:** Symmetric $Q \in \mathbb{R}^{n \times n}$ is positive semi-definite (PSD) with entries:

$$Q_{ij} = \phi(x_i)^\top \phi(x_j), \quad i, j = 1, \dots, n$$

- **Kernel function:**

$$K(x, z) = \phi(x)^\top \phi(z)$$

in this notation, entries of kernel matrix are

$$Q_{ij} = K(x_i, x_j)$$

- **Kernel methods:** (sometimes also called “kernel trick”)
 - ▶ algorithms/methods that use $K(x, z)$ and Q , and avoid using $\phi(x)$ or X directly

Another Example: Polynomial Kernel

- Consider $K(x, z) = (x^\top z)^2$
- $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^4$ is

$$\phi(x) = (x_1^2, x_1x_2, x_2x_1, x_2^2)$$

Example: Gaussian Kernel

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

aka *radial basis function*. Some observations:

- non-linear kernel with a lot of flexibility
- corresponds to an infinite dimensional ϕ (i.e., cannot implement the corresponding feature mapping ϕ).

Model Fitting by Regularized Least Squares

Consider the ℓ_2 -regularized Least Squares problem, aka **Ridge Regression**, or Tikhonov regularization:

$$\min_{\theta} \|X\theta - y\|_2^2 + \lambda \|\theta\|_2^2$$

- y is the n -vector with entries $y_1, \dots, y_n$
- we fit a model $f(x) = \theta^\top \phi(x)$ to data points $x_1, \dots, x_n, y_1, \dots, y_n$
- optimal solution: $\hat{f}(x) = \hat{\theta}^\top \phi(x)$ where

$$\hat{\theta} = (X^\top X + \lambda I)^{-1} X^\top y$$

From Primal to Dual

Start with regularized least squares in feature space:

Key Takeaway

This is the representer theorem in action.

Substitute into the objective

Primal vs Dual Views of Kernel Regression

Primal (Feature Space)

- Parameters $\theta \in \mathbb{R}^p$.
- Optimize:

$$\min_{\theta} \|y - \Phi\theta\|^2 + \lambda\|\theta\|^2.$$

- Solution:

$$\hat{\theta} = (\Phi^\top \Phi + \lambda I_p)^{-1} \Phi^\top y.$$

- Expensive if p (feature dim) is large or infinite.

Dual (Data Space)

- Parameters $\alpha \in \mathbb{R}^n$.
- Equivalent optimization:

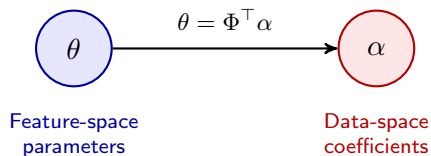
$$\min_{\alpha} \|y - K\alpha\|^2 + \lambda\alpha^\top K\alpha.$$

- Solution:

$$\hat{\alpha} = (K + \lambda I)^{-1} y.$$

- Works via the kernel matrix
 $K_{ij} = k(x_i, x_j)$ —no explicit ϕ needed!

Primal vs Dual Views of Kernel Regression



Connection: feature-space weights θ are expressed via data-space coefficients α .

Why Regularize in Kernel Methods?

Problem without regularization:

- Kernel methods (e.g., kernel ridge regression):

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x), \quad \alpha = K^{-1}y \quad (\text{if } \lambda = 0)$$

- Issues:
 1. **Overfitting:** Flexible kernels (like RBF) can interpolate training points perfectly but generalize poorly.
 2. **Numerical instability:** K may be nearly singular or ill-conditioned.

Effect of Regularization

Regularized kernel ridge regression:

$$\alpha = (K + \lambda I_n)^{-1}y, \quad f(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$$

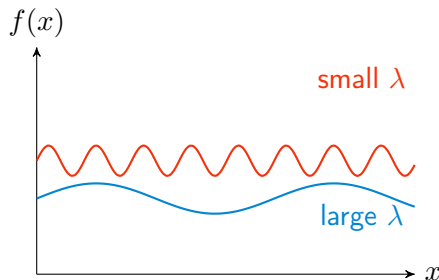
What regularization does:

- Adds λI_n to **stabilize** matrix inversion.
- Controls the **complexity/smoothness** of the function.
- Small $\lambda \rightarrow$ more flexible (can overfit)
- Large $\lambda \rightarrow$ smoother function, less variance

Regularization in Kernel Methods: Intuition

Key idea: Regularization controls function complexity in feature space.

- Kernel regression: $f(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$
- Regularized: $\alpha = (K + \lambda I_n)^{-1} y$
- λ penalizes the RKHS norm $\|f\|_{\mathcal{H}}^2$ (how "wiggly" f is)
- Small $\lambda \rightarrow$ flexible, risk of overfitting
- Large $\lambda \rightarrow$ smoother function, less variance



RKHS perspective:

$$\min_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2$$

Intuition

- Regularization balances **bias-variance tradeoff** in kernel space.
- Especially important for highly flexible kernels (e.g., RBF or high-degree polynomial kernels).
- Prevents wild oscillations and improves generalization.

Recap: Kernel Method

- Essence of method:
 - ▶ If we can write an algorithm only in terms of $\phi(x_i)^\top \phi(x_j)$, we don't need to explicitly enumerate $\phi(x_i)$ for $i = 1, \dots, n$
 - ▶ Instead can just compute $Q_{ij} = K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$
 - ▶ And can even handle infinite-dimensional feature maps $\phi(\cdot)$
- Examples:
 - ▶ Linear kernel $K(x, z) = z^\top x$
 - ▶ radial basis function $K(x, z) = \exp(-\gamma \|x - z\|^2)$
 - ▶ polynomial kernels $K(x, z) = (x^\top z + c)^p$
- Predictions only depend on training data through kernel function (which is just a dot product)